

# PBA: 4. User Defined Functions & Visualization II

Fall 2025

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

# Roadmap

## 1. Flexible data structures and user defined functions in R

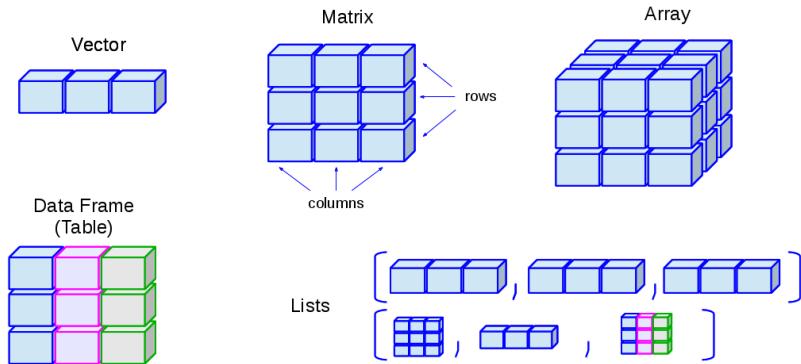
- Working with lists and data frames
- Creating and nesting functions
- Local vs. global variables
- Scoping rules in R
- Creating binary operators

## 2. Data Visualization

- Review of ggplot2 plotting system
- Histogram and boxplots
- Grouped data
- Principles of analytic graphics → *We will return to it after discussing causality in a couple of weeks!*

Rest of the class: In-class exercises for Visualization

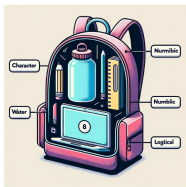
# Recall: Data Structures



- Vectors (1D), matrices (2D), arrays (nD) can store only **single data type** whereas lists (1D) and data frames (2D) can store **multiple data types**.

↪ nD = n Dimensional.

## Lists: 'Flexible' 1D Vectors



Source: created using a generative AI, DALLE-3!

### Prompt I Used to generate the images!:

Photo of an open backpack showing its contents: a pencil labeled 'character (c(''apple'', ''tree''))', a water bottle labeled 'numeric (c(1,2,3,4))', a laptop displaying 'logical (c(T,F,F))', and a calendar marked 'date-time (2023-10-18)'. Each item represents a different R data type.

# Introduction to Lists in R

- **Definition:** A vector that can house multiple data types, e.g., numeric, logical, strings.
  - Contains multiple components indexed with `$` or `[[ ]]` operators.
  - E.g., an employee database with name, salary, and union membership.
- Creating lists with `list()`

```
1 > j <- list(name="Joe", salary=55000, union=T); j # creating a list with component names
2 $name
3 [1] "Joe"
4 $salary
5 [1] 55000
6 $union
7 [1] TRUE
8
9 > jalt <- list("Joe", 55000, T); jalt #
10 [[1]]
11 [1] "Joe"
12 [[2]]
13 [1] 55000
14 [[3]]
15 [1] TRUE
16
17 > z <- vector(mode="list") # since a list in R is a vector!
```

# Accessing List Components

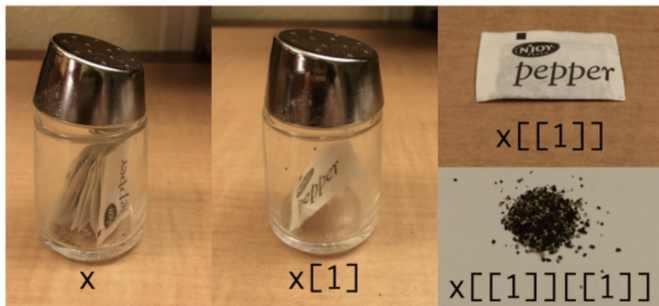
- We can point to specific component(s) of a list using the \$ or [[ ]] operators:

```
1 > j <- list(name="Joe", salary=55000, union=T)
2 > j$salary; j[["salary"]]; j[[2]] # All return 55000
3 [1] 55000
4 [1] 55000
5 [1] 55000
6
7 > class(j[[2]]) # Identifies the data type as 'numeric'
8 [1] "numeric"
```

- The [ ] operator can also be used, but this will return another list!:

```
1 > j["salary"]; j[2] # Both return a sub-list
2 $salary
3 [1] 55000
4
5 $salary
6 [1] 55000
7
8 > class(j[2]) # Identifies the output as a 'list'
9 [1] "list"
```

# List: Indexing Illustrated



Source: <https://stackoverflow.com/questions/1169456/the-difference-between-bracket-and-double-bracket-for-accessing-the-el>

- **Visual aid to the rescue!**

1. Imagine a bottle containing a bag of pepper grains.
2. In this analogy,
  - the bottle = list (X),
  - the bag(s) = component(s) (X[[1]]), and
  - the grains = elements within a given component (e.g., X[[1]][1]).
3. To get a specific grain, you first need to open the bottle, then open the bag.

# More Examples on Indexing Lists

- Let's think it through:

```
1 > y <- list(c(1,2,3), "abc"); y # Generate a list of length 2
2 [[1]]
3 [1] 1 2 3
4
5 [[2]]
6 [1] "abc"
7
8 > y[[1]][1]
9 [1] 1
10
11 > y[1][1]
12 [[1]]
13 [1] 1 2 3
```

- Q: What would R return if we ran the following codes?

```
1 > y[1][[1]][1] # What will R return?
2 > length(y) # what is the length of the list y?
```



# Modifying List Components in R

- We can edit components using the indexing & assignment operators.
- Example:

```
1 > z <- list(a = "abc", b = 12) # generate a list of length = 2
2 > z$c = "sailing" # add a third component named c
3
4 > z[4:5] <- c(F,T); z$b <- NULL; z # add 4th & 5th components then remove the 2nd component b
5 $a
6 [1] "abc"
7
8 $c
9 [1] "sailing"
10
11 [[3]]
12 [1] FALSE
13
14 [[4]]
15 [1] TRUE
16
17 # Using z[4:5] <- c(F, T) performs element-wise assignment, filling separate list components.
```

- Vector indexing with Lists:
  - Q: How do we remove the 2nd and the 3rd components of z altogether?
  - R allows vector indexing for `[]` operators! (With `[[ ]]`, vectors perform recursive indexing instead.)

# List as a Recursive Data Structure

- Lists can contain elements of any data type, and thus can also contain other lists as elements.
  - $\rightsquigarrow$  This property is known as 'recursiveness,' making lists in R a recursive data structure.
- **Example:**

```
1  b <- list(u = 5, v = 12)
2  c <- list(w = 13)
3
4  a <- list(b, c); a
5  [[1]]
6  [[1]]$u
7  [1] 5
8
9  [[1]]$v
10 [1] 12
11
12
13 [[2]]
14 [[2]]$w
15 [1] 13
16
17 length(a) # Length of list a is still 2!
18 [1] 2
```

## **Data Frames:** *'Flexible'* 2D Vectors

Or... think of an Excel sheet containing data where each column represents a **variable/feature** and each row represents an **observation**.

# Data Frame: Generation & Indexing

- As we did so far, we can manually generate a data frame:

```
1 > kids <- c("Jack", "Jill")
2 > ages <- c(2, 10)
3
4 > data <- data.frame(kids, ages, stringsAsFactors=F); data
5      kids ages
6 1   Jack    2
7 2   Jill   10
```

- And, point to specific values using the indexing operators, **but..**:

```
1 > data[[1]]; data$kids # list-like indexing
2 [1] "Jack" "Jill"
3 [1] "Jack" "Jill"
4
5 > data[,1] # matrix-like indexing
6 [1] "Jack" "Jill"
```

- It's much more common to **import data from external sources** rather than manually creating one!

# Data Frame: Importing Data

- Importing data in R using `read.table()` or `read.csv()`
  - Your choice of import method may vary depending on the file format of your data, such as text or CSV.

```
1 > exam1 <- read.csv(url("https://bit.ly/3ZqTQLY")); exam1
2 ID Exam1 Exam2 Quiz
3 1 1 3.3 2.0 3.7
4 2 2 4.0 4.0 4.0
5 3 3 2.3 0.0 3.3
6 4 4 2.2 1.0 3.3
7 5 5 3.1 1.2 3.9
8 > exam1 <- read.table(url("https://bit.ly/3ZqTQLY"), sep = ",", header = T)
```

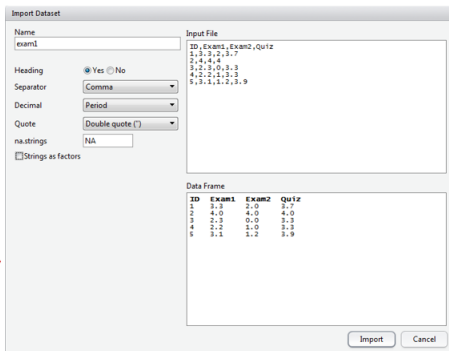
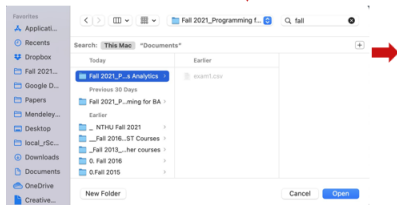
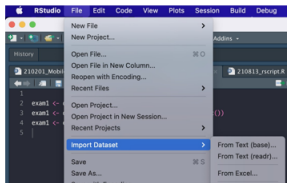
- If importing data from a local file:

```
1 # Use full file path:
2 > exam1 <- read.csv(file = "/Users/J1Yoo/Documents/Fall_2022_PBA/import_data.csv"); exam1
3
4 # Or..
5 > setwd("/Users/J1Yoo/Documents/Fall_2022_PBA") # set up the working directory,
6 > exam1 <- read.csv(file = "import_data.csv") # then use a relative file path.
```

- *Note:* use your file/directory paths when importing data from local files!

# Importing Data via RStudio Menu

- For those who prefer GUI, RStudio offers menu-based data import.
  - Advantage of Code-Based Import:** It creates an R script that serves as a reusable record, allowing for easy data re-importation in future sessions.



Auto. executes the func. below

```
> exam1 <- read.csv("~/Documents/Fall 2021_Programming for Business Analytics/
exam1.csv", header=FALSE)
> View(exam1)
```

**User-defined functions** in *R* enables us to work with data more efficiently, offering customized solutions for business challenges.

# Functions in R

- Recall that functions in R are objects which takes in input arguments then performs a specific task (i.e., `function(arglist) expr`).
- Examples:**

```
1 # Fahrenheit to Celsius function
2 f <- function(x) (x-32)/1.8
3 f <- function(x) {(x-32)/1.8}
4 f <- function(x) return((x-32)/1.8)
5 f(100) # Output: [1] 37.77778
6
7 # Celsius to Fahrenheit function
8 f <- function(x) x*1.8+32
9 f(37.7778) # Output: [1] 100
```



# Setting Default Values in Functions

- Default values in function arguments simplify function calls and improve code reusability.

```
1 > f <- function(a, b = 1, c = 2, d = NULL) {  
2   cat(a,b,c,d) # cat() concatenates values then prints it on screen  
3 }  
4  
5 > f(1)           # Uses default values for b, c, and d  
6 1 1 2  
7  
8 > f(1,2,3,4)     # Overrides default values  
9 1 2 3 4  
10  
11 > f(1,2,3)       # Partially overrides default values  
12 1 2 3  
13  
14 > f(3, c=5)      # Positional and named argument matching  
15 3 1 5  
16  
17 > f(d=7)         # Missing required argument 'a' leads to a runtime error
```

- **Tip:** Default values are particularly useful for:
  - Making functions more user-friendly (convenience).
  - Allowing optional customization (flexibility).
  - Avoiding missing argument errors (error prevention).

# Nesting Functions in R

- Functions can be nested within other functions to create more complex operations.

```
1 # Define function f that calls a nested function, g
2 f <- function(x,y){
3   print(x)
4   g <- function(y) { print(y) }
5   g(y)
6 }
7
8 f(1, 2)
9 [1] 1
10 [1] 2
```

- **Tip:** Use `args()` and `body()` to inspect a function's input arguments and code body.
  - Especially useful for understanding how complex functions work.

# Passing on Arguments to Other Functions

- The ... argument allows you to pass a set of arguments to another function.

```
1 # Define function f
2 f <- function(x, y) {
3   print(x)
4   print(y)
5 }
6
7 # Define function g that calls f
8 g <- function(z, ...) {
9   print(z)
10  f(...)
11 }
12
13 # Call function g
14 g(1, 2, 3) # Output: g prints 1, f prints 2 and 3
15 [1] 1
16 [1] 2
17 [1] 3
```

- **Key Takeaway:**
  - The ... parameter allows you to pass arguments through to other functions without explicitly naming each one.

# Understanding Variable Scope in R

- A **scope** refers to the region of code where a variable can be accessed or modified.
- **Types of Scopes:**
  - **Global scope:** accessible throughout the code.

```
1 global_var <- "I am global"
```

↪ Global variable is accessible from any part of the code, both inside and outside of functions.

- **Local scope:** the region within a function.

```
1 my_function <- function() {  
2   local_var <- "I am local"  
3 }
```

↪ Variables declared here are accessible only within that function. Each function call creates a new local scope.

# Example: Why Understanding Scope Matters

```
1 # Student wants to keep track of a total
2 total <- 0
3
4 add_to_total <- function(value) {
5   total <- total + value # Creates LOCAL total!
6   print(paste("Total is now:", total))
7 }
8
9 add_to_total(10) # Output: Total is now: 10
10 add_to_total(5) # Output: Total is now: 5 (not 15!)
11
12 print(total)    # Output: [1] 0 (Still zero!)
```

- What happens?
  - Without understanding scope, the student expects total to be 15, but it remains 0. Each function call creates a new local total instead of modifying the global one.
- Key takeaway:
  - Understanding scope prevents logical errors (bugs) caused by variables with the same name in different scopes.

# Function: Scoping Rules (1-2)

- **Rule 1. Global Variables:** Variables declared in console can be accessed anywhere.

```
1 n <- 1
2 f <- function () print (n)
3 f() # Output: [1] 1
4
5 n <- 2
6 f() # Output: [1] 2
```

- **Rule 2. Local Variables:** Variables declared inside a function are prioritized (shadowing/masking global variables with the same name).

```
1 n <- 100
2 f <- function () {n <- 1; print (n)}
3
4 f() # Output: [1] 1
```

# Function: Scoping Rules (3-4)

- **Rule 3. Local Variable Accessibility:** Variables declared inside a function cannot be accessed from outside.

```
1 rm(list=ls()) # Warning: this will remove everything in your R Session!
2
3 f <- function () x <- 1
4 f()
5 x # Error: object 'x' not found
```

- **Rule 4. Argument Priority:** Input arguments are prioritized over global variables. (Note: Arguments ARE local variables, thus Rule 4 demonstrates another angle on the shadowing rule.)

```
1 n <- 100
2 f <- function (n) print (n)
3 f(1) # Output: [1] 1
```

# Function: Scoping Rules (5)

- **Rule 5. Nested Functions:** The same scoping rules apply at each level. Variables are looked up by searching outward through enclosing scopes (nested scope lookup/chain).

```
1 f <- function (x) {  
2   a <- 2  
3   g <- function (y) {  
4     print (y + a)  
5   }  
6   g(x)  
7 }  
8  
9 a <- 100  
10  
11 f(1) # Q: what would R return?
```

- **Question:**
  - If we commented out `a <- 2` (line 2) from function `f`, what would happen?



# Function: Scoping Rules (6)

- **Rule 6. Super Assignment Operator:** Use `<<-` to modify or create a variable in the nearest enclosing (parent) environment where it exists.

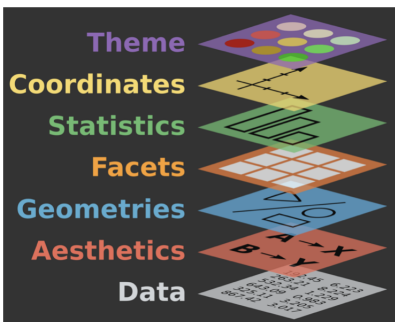
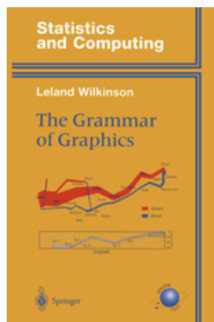
```
1  b <- 0
2  f <- function() {
3    a <- 1
4    g <- function(a, b) {
5      a <<- 2
6      b <<- 2
7      print(a)
8      print(b)
9    }
10   g(3,3)
11   print(a)
12   print(b)
13 }
14 f() # Output: [1] 3, [1] 3, [1] 2, [1] 2
```

- **Question:**
  - What if we were to use `<-` in place for `<<-`?
- **Caveat:**
  - Allows modifying variables in outer scopes. This is an advanced feature used sparingly (e.g., package development). For typical data analysis, have functions return values and explicitly assign them to variables.

## **Data Visualization II**

# ggplot2: The Grammar of Graphics

“...the grammar tells us that a statistical graphic is a mapping from data to aesthetic attributes (colour, shape, size) of geometric objects (points, lines, bars). The plot may also contain statistical transformations of the data and is drawn on a specific coordinate system” – Leland Wilkinson



Let's see how we can create histograms/boxplots and work with grouped data using ggplot2.

# Midwest Data Redux

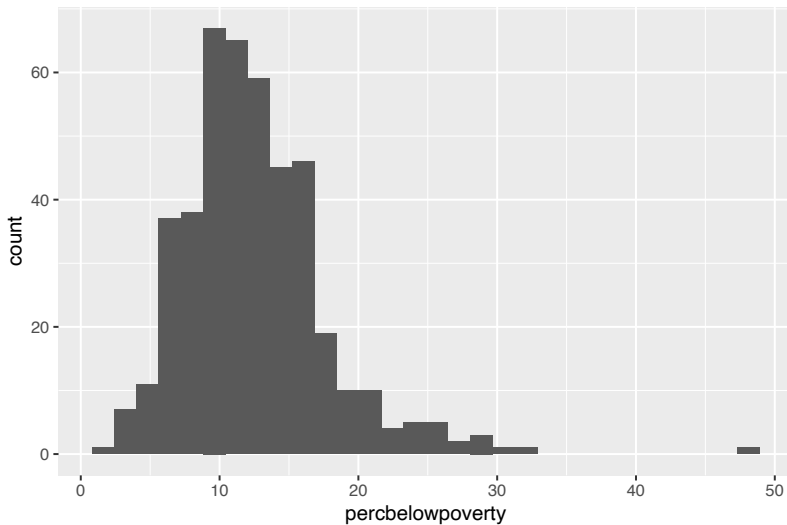
```
1 > ggplot2::midwest ## Install the package first using install.package() if you haven't already!
2
3 # A tibble: 437 × 28
4   PID county state area poptotal popdensity popwhite popblack popamerindian popasian popother
5   <int> <chr> <chr> <dbl> <int> <dbl> <int> <int> <int> <int> <int>
6 1 561 ADAMS IL 0.052 66090 1271. 63917 1702 98 249 124
7 2 562 ALEXAN IL 0.014 10626 759 7054 3496 19 48 9
8 3 563 BOND IL 0.022 14991 681. 14477 429 35 16 34
9 4 564 BOONE IL 0.017 30806 1812. 29344 127 46 150 1139
10 5 565 BROWN IL 0.018 5836 324. 5264 547 14 5 6
11 6 566 BUREAU IL 0.05 35688 714. 35157 50 65 195 221
12 7 567 CALHOUN IL 0.017 5322 313. 5298 1 8 15 0
13 8 568 CARROLL IL 0.027 16805 622. 16519 111 30 61 84
14 9 569 CASS IL 0.024 13437 560. 13384 16 8 23 6
15 10 570 CHAMPA IL 0.058 173025 2983. 146506 16559 331 8033 1596
16 # 427 more rows
17 # 17 more variables: percwhite <dbl>, percblack <dbl>, percamerindian <dbl>, percasian <dbl>,
18 # percother <dbl>, popadults <int>, perchsds <dbl>, percollege <dbl>, percprof <dbl>,
19 # poppovertyknown <int>, percpovertyknown <dbl>, percbelowpoverty <dbl>,
20 # percchilbelowpovert <dbl>, percadultpoverty <dbl>, percelderlypoverty <dbl>, inmetro <int>,
21 # category <chr>
22 # Use `print(n = ...)` to see more rows
```

# Histograms

- **Histograms** show where there are more or fewer observations of a numeric variable.

```
1 require(ggplot2)
2 ggplot(data = midwest,
3       mapping = aes(x = percbelowpoverty)) +
4   geom_histogram()
```

- Split up range of variable into bins, count how many are in each bin.
- y aesthetic calculated automatically.



**Take away:** a majority of the counties have around 10% of their population living below the poverty line.

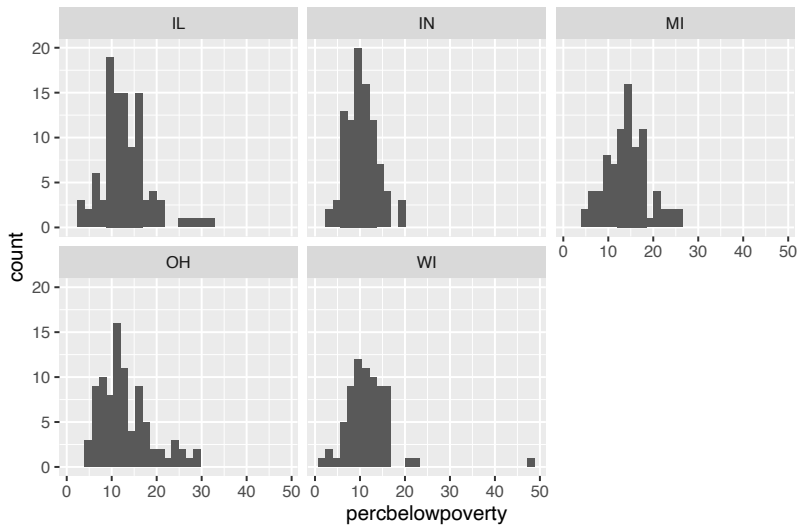
# Creating Small Multiples with Facets

- Small multiples: a series of similar graphs with the same scale/axes to help with comparing different partitions of a dataset

```
1 ggplot(data = midwest,  
2       mapping = aes(x = percbelowpoverty)) +  
3   geom_histogram() +  
4   facet_wrap(~ state)
```

- The variable used for faceting/splitting must be placed on the right-hand side of the tilde (~) operator
  - We'll see more of the ~ variable syntax (called a formula) throughout the semester!

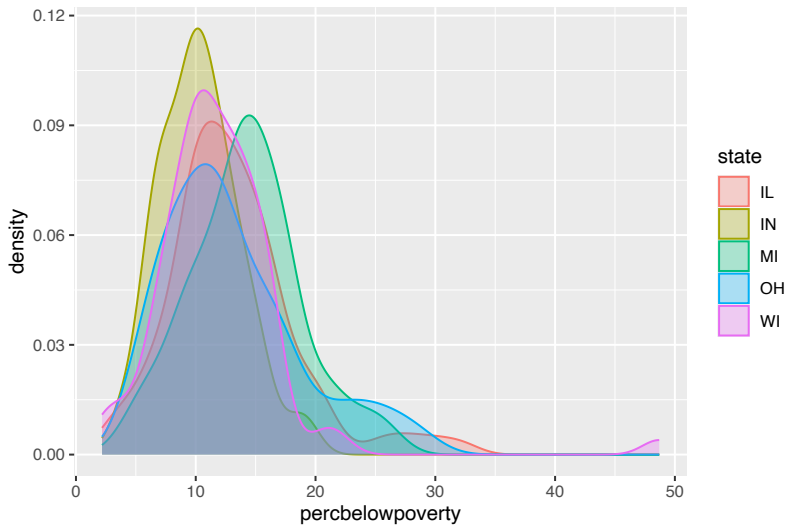




# Density as Alternative to Histogram

- A kernel density plot is a smoothed version of a histogram and slightly easier to overlay.

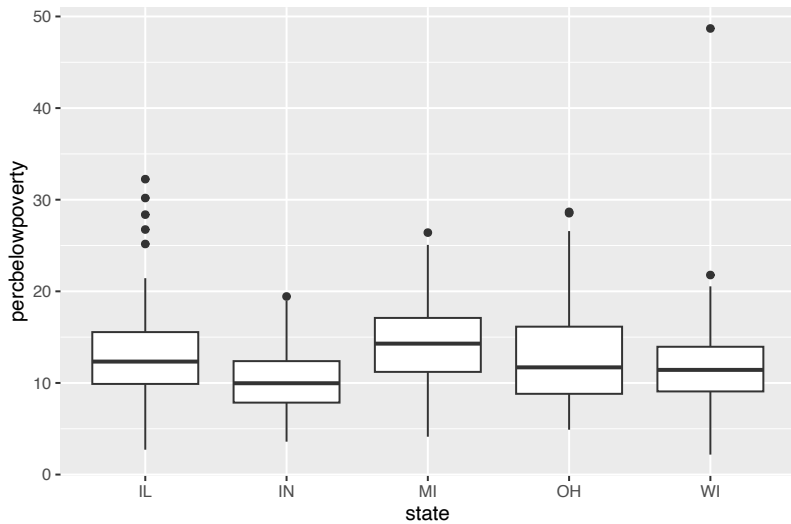
```
1 ggplot(data = midwest,  
2       mapping = aes(x = percbelowpoverty,  
3                     fill = state, color = state)) +  
4   geom_density(alpha = 0.3)
```



# Boxplots

- Boxplots are another way to compare distributions across discrete groups.

```
1 ggplot(data = midwest,  
2       mapping = aes(x = state,  
3                     y = percbelowpoverty)) +  
4   geom_boxplot()
```



# More on Boxplots

- “Box” represents middle 50% of the data.
  - 25% of the data above the box, 25% below
  - Width of the box is called the inter quartile range (IQR)
- Horizontal line in the box is the median
  - 50% of the data above the median, 50% below
- “Whiskers” represents either:
  - $1.5 \times \text{IQR}$  or max/min of the data, whichever is smaller
  - Data points beyond whiskers are outliers

Visualizing **grouped data** accurately is key to conveying the right insights and avoiding misleading interpretation.

# Gapminder Data

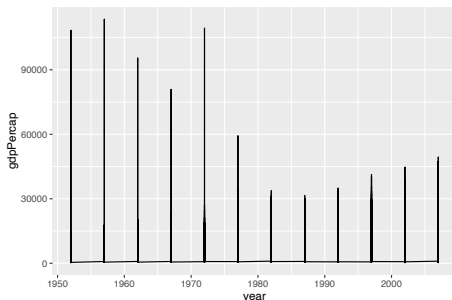
```
1 require(gapminder)
2 dplyr::glimpse(gapminder)
3
4 ## Rows: 1,704
5 ## Columns: 6
6 ## $ country   <fct> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", "Afgha...
7 ## $ continent <fct> Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, ...
8 ## $ year      <int> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, 2002, ...
9 ## $ lifeExp   <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.822, 41. ...
10 ## $ pop       <int> 8425333, 9240934, 10267083, 11537966, 13079460, 14880372, 12881816. ...
11 ## $ gdpPercap <dbl> 779.4453, 820.8530, 853.1007, 836.1971, 739.9811, 786.1134, 978.01. ...
```

- Let's say we are interested in learning about the trend in income in different countries.
  - ↪ Use `geom_line()` to create a line plot.



# Let's plot the trend in income.. wait what?

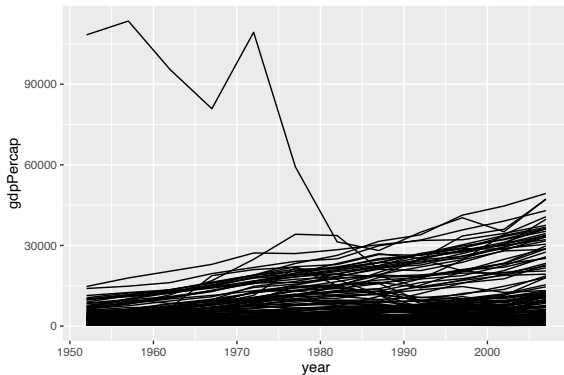
```
1 ggplot(data = gapminder,  
2       mapping = aes(x = year,  
3                     y = gdpPercap)) +  
4   geom_line() # Didn't specify how to group the lines..
```



- `geom_line()` connected points from different countries in the same year.
  - What we wanted was to connect the points from same countries in different years (i.e., time trends)!

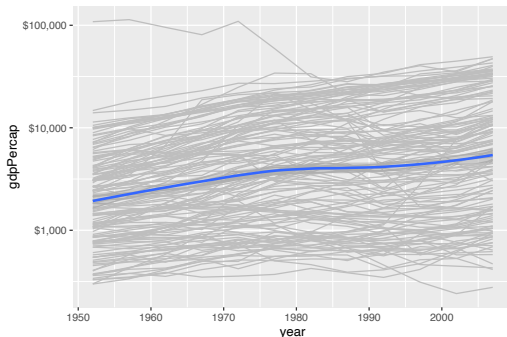
# Tell `geom_line()` how to group the lines

```
1 ggplot(data = gapminder,  
2       mapping = aes(x = year,  
3                     y = gdpPercap)) +  
4   geom_line(mapping = aes(group = country))
```



# Adjusting scales and adding a trendline

```
1 ggplot(data = gapminder,  
2       mapping = aes(x = year,  
3                     y = gdpPercap)) +  
4   geom_line(mapping = aes(group = country), color = "gray") +  
5   geom_smooth(method = "loess") +  
6   scale_y_log10(labels = scales::dollar)
```



Q: What do we observe? Why do you think this is the case? (innovation? capital accumulation?)

## On to the In-class Exercises!

*Contact Information:*

[jaewon.yoo@iss.nthu.edu.tw](mailto:jaewon.yoo@iss.nthu.edu.tw)

<https://j1yoo.github.io/>



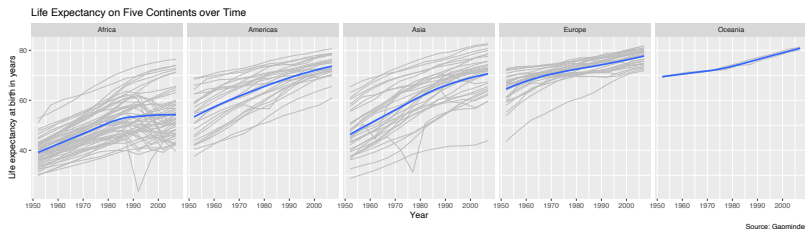
## **In-Class Exercises**

# In-Class Exercises: Background

- We want to find out the answer to the following question:
  - Q: how does life expectancy on five continents (e.g., Asia, Europe, Americas) change over time?
- The gapminder dataset contains some information about life expectancy across different continent, thus we could visualize the data to answer the question.
  - Some relevant variables:
    - country: name of the country
    - continent: name of the country's continent
    - year: year of the measurement, ranging from 1952 to 2007 in 5-year increments
    - lifeExp: life expectancy at birth, in years

# In-Class Exercises: Task

- Recreate the following plot:



- The plot shows each country's life expectancy trajectory over time, broken out by continent with smoothed average lines overlayed for each continent.
  - To get started, we'll give you a few clues about what was done to create the plot in the following slide.

# In-Class Exercises: Instructions

- The lines for each country use the color "gray"
- The size of the smoothed line is 1.1 and the method used is the loess smoother.
- Turn off the standard errors.
- Make sure that the facets are all on one row. Look at the `facet_wrap` documentation if need help with this.
- Make sure that the labels are correctly specified.

Q: What does the visualization reveal? Explain what you observe verbally.